

DraftPilot AI — Project Overview

The "what" and "why" doc. Read this first.

- [DraftPilot_Project_Overview.md](#) (you are here) — what we're building, who it's for, why each piece exists
- [DraftPilot_Final_Vision.md](#) — what the system looks like at month 6 (the destination)
- [DraftPilot_Build_Guide.md](#) — step-by-step path to get there

1. What is DraftPilot AI, in one paragraph

DraftPilot AI is a live fantasy football draft assistant. A user opens it before or during their fantasy draft, connects their league (Sleeper, eventually Yahoo), and the app tells them — **for their specific roster, their specific scoring rules, this specific draft slot, and this exact moment in the draft** — which player to pick, why, and what's likely to happen if they wait. The recommendation combines projected player value, uncertainty, league context, and thousands of simulated future picks. A grounded LLM advisor sits on top, answering natural-language questions ("should I take a WR or wait for RB?") using the same underlying data, never inventing numbers.

2. The problem we're solving

The user

A fantasy football manager preparing for or in the middle of a draft. They have:

- 60–90 seconds per pick
- A specific roster being built (positional needs evolving every pick)
- League-specific scoring (PPR, half-PPR, custom) that changes player value
- A draft slot that constrains what picks they get
- A vague sense of who the "good" players are but no way to think in real time about *who falls back to them*

What's broken about the existing options

- **Static rankings** (FantasyPros, ESPN cheat sheets) are pre-computed lists that ignore your draft slot, your roster, and what's left on the board.
- **Existing tools** that adapt to draft state are mostly closed, paid, and don't show *why* they recommend what they do.
- **None of them** model *future picks*. They tell you the best available player now; they don't tell you "this WR will likely be available next round, but no RB of this tier will."
- **None of them** combine grounded LLM reasoning. ChatGPT-style answers hallucinate stats. Static tools can't answer "compare these two strategies."

What DraftPilot does differently

- It models the *whole draft*, not just the current pick.

- It exposes *uncertainty* (floor / median / ceiling, fall-through probability) instead of pretending projections are precise.
- It explains *every recommendation* with structured reason codes.
- It lets you *ask questions in natural language* and get answers grounded in actual model outputs — no hallucinated stats.

That's the product. The next sections explain how each technical layer earns its place in serving that product.

3. The intelligence layers (and what each one earns)

DraftPilot's "brain" is four layers stacked. Each layer answers a specific question. Removing any one of them removes a core capability.

Layer 1 — Player projections (with uncertainty)

The question it answers: "How many fantasy points will this player score, and how confident are we?"

What it produces: For every player, five numbers:

- **projected_points** — the expected (median) outcome
- **floor** — the 15th-percentile outcome ("the bad case")
- **ceiling** — the 85th-percentile outcome ("the great case")
- **bust_probability** — chance they underperform their ADP meaningfully
- **breakout_probability** — chance they significantly exceed expectation

Why it has to exist: A recommendation engine without projections has nothing to recommend on. A recommendation engine with only a single projected number can't tell the difference between "safe pick" and "upside swing," which is half the value of a draft assistant.

Why the uncertainty matters: Every fantasy projection is wrong. Some are wrong predictably (a steady veteran), some unpredictably (a rookie with high variance). Hiding that uncertainty produces overconfident picks. Exposing it lets the user choose risk consciously.

Build progression:

- MVP: a simple heuristic ("last year's points adjusted for age + position trend"). Wrong, but plausible — and good enough to unblock everything downstream.
- Month 3+: real LightGBM models trained on multi-season nflverse data, quantile regression for floor/ceiling, calibrated classifiers for bust/breakout, all backtested season-over-season against an ADP baseline.

The cold-start problem (rookies): The heuristic "last year's points" has nothing to say about rookies — who are simultaneously the *highest-variance* and *most-discussed* players in any draft, and disproportionately what users ask the advisor about. The MVP must explicitly handle the no-prior case: fall back to a position-and-draft-capital prior (NFL draft round → expected rookie-season range by position) rather than projecting zero or crashing. This is a named MVP requirement, not a Month-3 refinement, because a draft tool that silently mis-handles the rookies people are debating loses credibility on pick one.

Layer 2 — Draft optimization (the scoring function)

The question it answers: "Given my roster, my league's scoring, the available players, and where I am in the draft — what's this player worth to me, *right now*?"

What it produces: For every available player, a single score (and a breakdown of the components that produced it):

```
score = projection_value
       + value_over_replacement
       + roster_need_bonus
       + adp_value_bonus
       + tier_scarcity_bonus
       - risk_penalty
       - reach_penalty
```

Why it has to exist: Projections alone don't recommend picks. A QB with 320 projected points isn't necessarily better than an RB with 280 projected points — it depends on positional scarcity, your league's QB premium, what you've already drafted, and how good the next-best QB will be. The scoring function turns "absolute player value" into "value to *this user right now*."

Why each component:

- **Projection value** — anchors everything to actual production.
- **VORP** (value over replacement) — a 280-point RB is worth more than a 280-point QB because replacement-level RB scores ~120 and replacement-level QB scores ~220. This is the single most important fantasy concept.
- **Roster need bonus** — once you have your starting RBs, the marginal RB matters less than your first WR.
- **ADP value** — taking a player 30 picks before their market price ("reaching") usually means you could have waited and gotten them. Taking a player 30 picks after their market price ("steal") is upside.
- **Tier scarcity** — within a position, players cluster into tiers of similar value. Picking from the *last* player in a tier matters more than picking from the middle of one.
- **Risk penalty** — a player with a current injury, depth chart battle, or holdout is worth less than their raw projection says.
- **Reach penalty** — discourages obvious overdrafting.

Build progression:

- MVP: hand-coded function with hand-tuned weights, exhaustively unit-tested. Strategies (Balanced, Safe, Upside) are just different weight presets.
- Later: more sophisticated weight schemes informed by historical "good draft" patterns.

Layer 3 — Monte Carlo draft simulator (the centerpiece)

The question it answers: "If I take this player now, what does my roster *probably* look like after my next 2–3 picks? And how does that compare to taking someone else now?"

What it produces: For every candidate pick, a distribution of future-roster outcomes — represented in the UI as:

- **expected_roster_value** (the headline ranking metric)
- **chance_available_next_pick** for each alternative
- **tier_drop_risk** per position
- Example simulated future rosters

Why it has to exist: This is the differentiator. Without it, DraftPilot is just "static rankings, but slightly smarter." With it, DraftPilot answers questions no static tool can:

- "If I take this WR now, will my top RB target survive to my next pick?"
- "Is there a tier cliff at TE before I pick again?"
- "Which of these three equally-rated players is the *safest* bet given draft dynamics?"

How it works conceptually:

1. Fork the draft state with the candidate player drafted.
2. For each pick between now and your next turn, *sample* an opponent pick — based on ADP, randomness, and opponent roster needs.
3. At your next turn, see what's the best player available.
4. Repeat thousands of times with different random seeds.
5. Aggregate into probabilities and expected values.

Why this is technically meaningful: It's pure-function, deterministic given a seed, vectorized for speed, and *defensible in an interview*. You can hand someone a 50-line Python snippet that reproduces the recommendation. That's the level of rigor that separates "shipped a project" from "built a real system."

Why the seeding matters: Two reviewers loading the same draft state should see *identical* recommendations. Without seeded randomness you can't reproduce a result, can't test it, and can't trust it.

The opponent model is the whole ballgame — and must be validated, not just shipped. The simulator is the differentiator, which means its *opponent model* (how it samples other teams' picks) carries the entire load. A "softmax over ADP rank with roster-need adjustment" can produce confidently wrong **chance_available_next_pick** numbers, and a fantasy-savvy viewer notices instantly when the sim claims a player will fall who obviously won't. So the opponent model ships with an explicit **calibration plan**, not just a heuristic:

- Backtest it against real historical Sleeper drafts (replay each draft pick-by-pick, ask the model "is player X available at my next pick?", compare to ground truth).
- Report calibration: when the sim says 70% available, it should be available ~70% of the time. A reliability curve (predicted vs. actual) is the headline artifact.
- This calibration chart is what converts "cool simulation" into "trustworthy simulation" — and it is itself one of the strongest things to show a reviewer. An uncalibrated simulator is worse than no simulator, because it's confidently wrong.

Build progression:

- MVP: simple opponent model (softmax over ADP rank with roster-need adjustment), 2000 trials, <800ms for the simulation step. Ships *with* the calibration backtest above — the backtest is part of the MVP, not a later refinement.
- Later: learned opponent archetypes from historical draft data, opponent strategies, parallelized trials.

Layer 4 — LLMs as structured-intelligence + conversational surface

This layer has two distinct jobs. Most products with an "AI chat" misunderstand which job is doing the real work.

4a (primary) — News-to-structured-signal extraction

The question it answers: "What just happened in the NFL that should change how we value players?"

What it produces: Structured `player_event` records derived from unstructured news:

```
{
  "player_id": "...",
  "event_type": "injury | role_change | depth_chart | suspension | holdout | coach_quote",
  "severity": "minor | moderate | major | season_ending",
  "impact_direction": "negative | positive | neutral",
  "confidence": 0.0-1.0,
  "source_url": "...",
  "published_at": "..."
}
```

Why this is the most important LLM use in the system: Every recommendation gets smarter because of it. A draft tool that ignores breaking news will recommend a player who got injured yesterday. The LLM here turns a brittle, never-quite-right rule/regex parsing problem into a reliable structured-output problem — and **the structured output feeds directly into deterministic code** (the `risk_penalty` in the scoring function). The LLM produces a signal; the scoring function uses it. The LLM never moves a player up or down a ranking.

Why this is the right job for an LLM:

- Player news is unstructured prose written by hundreds of different sources in inconsistent styles. Regex/NER pipelines for this are notoriously fragile.
- The output is a small, strict JSON schema — exactly the kind of task modern LLMs do reliably with structured output / tool-use APIs.
- It's a batch job, so latency is irrelevant and cost is minimized by using a small, cheap model.
- It's invisible to the user, so it delivers value whether or not anyone opens the chat surface.

Build progression:

- MVP: a small, opinionated set of news sources; one extraction prompt per article; events stored in `player_events`.
- Later: RAG-indexed news with pgvector, multi-pass extraction with self-checking, per-source quality scoring, more event types.

4b (secondary) — Conversational advisor for explanation and open-ended questions

The question it answers: "I want to *talk to* the tool about my draft — explain, compare, what-if."

What it produces: Natural-language answers to questions like:

- "Why did you recommend Y over Z?"
- "Compare zero-RB to hero-RB for my draft slot."
- "What's the latest on Player X and how does it change his value?"
- "Should I take a WR now or wait for RB?"

Why this exists (and what it's *not* for): The other three intelligence layers produce structured numbers and reason codes. The war room UI surfaces them as cards, charts, and badges. The advisor exists for the cases the static UI can't precompute — open-ended comparison, onboarding for non-expert users, and the "Why?" affordance attached to every recommendation. It is **not** the place to ask the LLM to rank players. That would re-introduce hallucination and break reproducibility.

The architectural rule that governs both jobs: *The LLM never invents numbers, never overrides the scoring function, and never decides a recommendation.* It either (a) produces a structured signal that deterministic code consumes (job 4a), or (b) explains the output of deterministic code in natural language (job 4b). This is the single most important rule in the system and the thing that keeps the project rigorous.

How grounding is enforced for the advisor:

- The advisor only has access to specific tools (`get_recommendations`, `run_simulation`, `compare_players`, `search_news`, etc.) that return structured data from the same backend the UI uses.
- The system prompt explicitly forbids inventing stats and requires citing tool outputs.
- A pre-response validator flags numeric claims not present in any tool result.
- Every advisor call is logged in `agent_traces` with its tool calls, outputs, and token cost — so any number in a response can be traced back to its source.

Build progression:

- MVP: single advisor endpoint with 5–6 tools, basic chat UI in the war room.
- Later: `search_news` tool over the pgvector index, "Why?" affordance pre-loaded with pick context, trace UI showing tool calls.

Why these are *not* "multi-agent"

The original design doc described "five specialist agents" (Draft Strategy, Projection, News/Injury, Market/ADP, Explanation). Re-reading that with the architectural rule above, four of those five are just *tool calls* against the existing backend — not agents that need to coordinate or negotiate. Only the explanation and the conversation involve actual LLM reasoning. So DraftPilot uses LLMs in **several distinct places**, each with its own prompt, model tier, and job — but they don't talk to each other. They share data through the database, not through messages. That's "orchestrator + tools," not multi-agent. The flat architecture is cheaper, faster, more deterministic, easier to test, and easier to defend in an interview than a multi-agent claim would be.

4. The full system, mapped to its purpose

Every box in the architecture diagram earns its place. Here's why each one exists *in service of the user-facing product*.

Component	Why it exists for the product
Next.js frontend	The product is a live draft tool. The user needs a polished, fast, interactive UI. Anything less than a real frontend means it's a notebook, not a product.
FastAPI backend	The brain. Hosts the scoring function, simulator, projection inference, LLM tool endpoints, and external-API integrations. Python because the ML and LLM code live in the same language — no cross-service serialization.
PostgreSQL	The whole product is built on relational data (players, leagues, drafts, picks, rosters, projections). Postgres + pgvector also gives us our embedding store for RAG without a second datastore.
Redis	Draft state is hot data — read on every UI tick, recomputed on every pick. Redis caches it. It also holds rate-limit counters, simulator result caches, and SSE pub/sub for real-time updates.
Background worker	Some work is too slow or too scheduled for a request handler: ingesting nflverse data, retraining models, pulling news, extracting events. Worker process handles it on its own clock.
Sleeper / Yahoo integrations	The data has to come from <i>the user's actual league</i> . A draft tool that requires manual roster entry would never be used. Sleeper is read-only and free (MVP), Yahoo is OAuth and demonstrates auth/security work.
nflverse historical data	Training data for projection models. Without multi-season usage and play-by-play, you have no features.
News ingestion + LLM event extraction	Player values change with injuries, role changes, holdouts. A draft tool that ignores breaking news will recommend players who got injured yesterday. This is also the <i>highest-leverage</i> LLM use in the system: turning unstructured prose into typed signals that deterministic code consumes.
LLM advisor chat	A secondary surface for open-ended questions, comparisons, and explanation that the structured UI cannot precompute. Never the source of numerical truth — always grounded in tool outputs.
MLflow	The projection models are not a one-time training. We retrain on a schedule, want to compare candidate models to current production, and need to roll back if a new model regresses. MLflow is the registry that makes this safe.
AWS + Terraform	The product needs to be live and demoable. Terraform makes the infra reproducible, teardownable, and reviewable as code — which is itself a major engineering signal.
CI/CD	We're shipping continuously. Every PR runs the full test suite and every merge deploys. Without this, the project becomes "works on my machine" and dies.
Observability	When the recommendation endpoint takes 4 seconds instead of 400ms, we need to know <i>why</i> . Structured logs + traces + Sentry + CloudWatch turn "the demo is broken" from a mystery into a 10-minute debug.

If you can't justify a component by mapping it back to the user-facing product, it shouldn't be in the system. This list intentionally excludes Kubernetes, microservices, Spark, Kafka, gRPC, and a separate vector DB — none of which earn their complexity for this product.

4a. How DraftPilot uses LLMs (concretely)

Most product designs that mention "AI" or "agents" gloss over *where exactly* LLMs are used and what jobs they do. Here is the explicit list for DraftPilot. The mental model is **one LLM-using application with several distinct jobs**, not "multiple LLMs talking to each other."

Job	When it runs	Model tier	Output	User-visible?
News relevance classifier	Background, every ~15 min on new articles	Cheap/fast (Haiku-class)	Per-article: is this fantasy-relevant? Which player(s)?	No
Structured event extractor	Background, immediately after relevance pass	Cheap/fast (Haiku-class)	One or more player_event JSON records per article, strict schema	No (the <i>effect</i> is visible — recommendations shift)
News embeddings	Background, on new articles	Embedding model (not an LLM)	Vector stored in pgvector for RAG retrieval	No
Advisor chat	Real-time, per user message	Smart (Sonnet/Opus-class)	Natural-language answer with cited tool outputs	Yes — chat panel in the war room
Recommendation explainer	On-demand, per "Why?" click	Smart (Sonnet-class)	Short prose paragraph rendered from structured reason codes	Yes — expanded on a card

Six points worth being explicit about:

- Different jobs, same code module.** All of these share one **services/llm/** module — one client wrapper, one logging path, one cost tracker. They are not five microservices and they are not five "agents." They are five functions that call the same client with different prompts and tools.
- Model tiering is on purpose.** Batch extraction work uses the cheapest model that produces reliable structured output. The user-facing advisor uses a smarter, more expensive model because it's reasoning over comparisons and intent. Putting Opus on news extraction would burn 10x the cost for no quality gain; putting Haiku on the advisor would visibly hurt response quality.
- The two batch jobs (relevance + extraction) are by far the highest *product* value.** They make every recommendation smarter, silently, on every user. The chat is by far the highest *demo* value. Both belong, but they earn their place for different reasons.

4. **The chat advisor calls the same tools the UI does.** `get_recommendations`, `run_simulation`, `compare_players`, `get_roster_needs`, `get_available_players`, `search_news`. There is no "advisor-only" backend logic. The advisor is a thin LLM layer over the same API that powers the war room. This is what keeps it grounded.
5. **News influences recommendations *only* through structured event records.** The LLM never tweaks a player's score with prose. It writes a row into `player_events`; the scoring function reads it. This is the firewall that prevents hallucinated numbers from leaking into recommendations.
6. **Everything is traced.** Every LLM call — batch or chat — writes to `agent_traces` with the prompt, tool calls, outputs, latency, and token cost. This is what makes the system debuggable, evaluable, and defensible in an interview.

Why this isn't "multi-agent orchestration"

These jobs don't coordinate with each other. The news extractor doesn't ask the advisor anything; the advisor doesn't negotiate with the explainer. They share data through the database (`player_events`, `player_projections`, `recommendations`), not through inter-agent messages. There's no orchestrator deciding which agent to invoke next. The flow is just:

```

worker schedule → relevance classifier → extractor → DB
                                                    ↓
user request → API → recommendation (reads DB) → response
                                                    ↓
                                                    (optional) advisor or
explainer → response

```

That's a system that *uses LLMs as components*, not a multi-agent system. Calling it "multi-agent" would be a bigger claim than the architecture supports — and a sharp interviewer would catch it. The flat orchestrator-plus-tools pattern is cheaper, more deterministic, easier to test, easier to evaluate, and (importantly) *honest about what it actually is*.

What this looks like in resume language

Designed an LLM layer with two distinct roles: (1) a batch extraction pipeline that converts unstructured player news into typed `player_event` records that feed deterministic risk scoring, and (2) a grounded tool-calling advisor that explains recommendations and answers open-ended draft questions using the same backend endpoints as the UI. Per-job model tiering (small models for batch, larger for user-facing), pgvector-backed RAG for news context, structured-output validation, full trace logging with token-cost attribution, and a strict no-hallucinated-numbers contract enforced by output validation.

That's a bullet a hiring manager can dig into for thirty minutes — because every claim maps to code in the repo.

5. Why this is built in this order

The build guide's phase order isn't arbitrary. Each phase exists to unlock the next, and the order is optimized to **maximize the chance you finish**.

Phase	What it unlocks	What dies without it
1. Walking skeleton	Everything downstream. Request goes browser → backend → DB and back.	Without this, every later "feature" is decorating a non-existent product.
2. Data spine	A place to put real data. Tables, migrations, seed data, first real endpoint.	Without this, you can't display anything meaningful in the UI.
3. Sleeper integration	Real users with real leagues.	Without this, the product can only show fake data and is never demoable.
4. Draft war room UI	The user-facing product shape.	Without this, you have an API nobody can use.
5. Recommendation engine (no ML yet)	The first interesting code. Heuristic projection + scoring function.	Without this, the war room renders data but recommends nothing.
6. Monte Carlo simulator	The technical centerpiece. The "this is a real system" moment.	Without this, the project is "league import + static rankings." Cool but not impressive.
7. LLM advisor v1	The "modern AI" surface. Grounded chat.	Without this, the project misses the GenAI signal that's currently table stakes.
8. Deploy the MVP	A public URL anyone can use.	Without this, the project is invisible — hiring managers never see it.
9. Production-grade backend	Auth, rate limits, logging, errors.	Without this, the project looks like a student exercise instead of a product.
10. Advanced ML	Real, backtested models.	Without this, the projections remain heuristics and the "ML" resume bullet is weak.
11. RAG + news	News-aware risk scoring.	Without this, recommendations are blind to recent reality.
12. AWS + IaC + MLOps	The cloud engineering signal.	Without this, the deployment is on a PaaS and the infra resume bullet is generic.
13. Yahoo OAuth + polish + docs	Security depth + simulator + the finishing touches that make the project feel done.	Without this, the project is "almost there but rough."

The v1 stop line (the most important line in this doc)

The single biggest risk to this project is not that it's badly designed — it's that it's *big*, and big solo projects die in the middle far more often than they ship. The table above lists 13 phases as if they're equal commitments. **They are not.** Treat **Phase 8 as a bright "v1 stop line": a complete, shippable, genuinely impressive product on its own.**

By the end of Phase 8 you have: a deployed public URL, real league import (Sleeper), a live draft war room, a heuristic-but-correct recommendation engine, the Monte Carlo simulator (the differentiator) *with its calibration backtest*, and a grounded LLM advisor v1. That is already a product worth demoing and a resume bullet worth defending. If life intervenes after Phase 8, you have shipped — not abandoned.

Phases 9–13 (production hardening, advanced ML, RAG/news, AWS+IaC+MLOps, Yahoo OAuth) are **bonus rounds that deepen specific signals, taken one at a time, each shippable on its own.** They are not a five-phase block you must clear to "finish." Re-frame your own sense of "done" around Phase 8, and the completion odds go up sharply. Everything past it is upside, not obligation.

The big inversion

Notice that **ML appears at Phase 10, not Phase 1.** Most students do this in the wrong order: they train a model first, then can't figure out where to put it. By the time you start training real models, the product knows exactly how to consume them, what shape the output needs to be, and how to fall back if a model fails. The model becomes a swappable component instead of the centerpiece. That inversion is the single most important pacing decision in this project.

Why each phase is sized this way

- Phases are roughly one PR per sub-step, so the PR history is granular and reviewable.
- Each phase ends with a *demoable state*. If you stop after any phase, you have something to show.
- "Boring" infrastructure phases (1, 2, 8, 9, 12) come *before* the dependent "interesting" phases (5, 6, 7, 10, 11) — because the reverse order causes painful retrofits.

6. What each piece of the build serves, concretely

This is the same mapping again, but from the *build guide* perspective: when you're staring at Phase X wondering "why am I doing this," look here.

You're doing	You're building	Because the product needs
Docker Compose	A reproducible local environment	A way to develop and demo without "works on my machine"
Alembic migrations	Versioned schema evolution	A database that can change safely as the product grows
player_id_mappings table	Cross-source ID reconciliation	Sleeper, nflverse, and ADP sources all use different player IDs — without mapping, the data doesn't join
Sleeper client + caching	A read-only door into real user data	Real leagues to import; rate-friendly behavior so you don't get blocked

You're doing	You're building	Because the product needs
TanStack Query	Server-state caching on the frontend	A war room that feels responsive instead of refetching everything constantly
Generated TS types from OpenAPI	Compile-time contracts between frontend and backend	Catching API drift before runtime; no silent type bugs
Replacement value + VORP	The single most important fantasy number	Knowing a 280-point RB is worth more than a 280-point QB
Scoring function components	The interpretable ranking logic	Reason codes ("Tier cliff at RB") that make every pick explainable
Seeded Monte Carlo simulator	Future-pick probability estimation	The "will this player fall to me?" question that no static tool answers
LLM structured extraction	News prose → typed <code>player_event</code> records	Recommendations that react to injuries, role changes, holdouts without brittle parsing
LLM tool-calling (advisor)	Grounded conversational explanation	A chat UI that doesn't hallucinate stats
Per-job LLM model tiering	Cheap models for batch extraction, smart models for user-facing chat	Cost control + appropriate quality per job
Pydantic everywhere	Boundary validation	Defense against external-API garbage and bad client input
Structured logs + request IDs	Per-request observability	Debugging "recommendation took 4s" in production
MLflow + promotion gates	Safe model evolution	Replacing the projection model without breaking the product
pgvector + RAG	Searchable news context	The advisor answering "what's the latest on Player X"
Structured event extraction	Deterministic news → risk signal	News influencing recommendations <i>without</i> letting the LLM tweak scores
Terraform	Infrastructure as reviewable code	A demo URL that can be torn down and rebuilt in 10 minutes; resume signal
GitHub Actions CI/CD	Continuous delivery	"Merge to main" → "deployed within 10 minutes"
Branch protection + PR templates	Process discipline	A repo that <i>looks</i> like a real team's repo to reviewers
ADRs	Decision archaeology	Future-you (and reviewers) understand <i>why</i> choices were made

You're doing	You're building	Because the product needs
OpenAPI versioning	Forward-compatible API evolution	Changing endpoints without breaking the frontend
Rate limiting on LLM endpoints	Cost control	Not waking up to a \$300 OpenAI bill
Drift monitoring	Model freshness signal	Knowing when retraining is actually needed

If you ever find yourself building something not on this list — pause. Either the list is incomplete (add it and explain why) or you're scope-creeping.

6a. Scope: what's in, what's deferred, and why

A project this big has dozens of "production-grade" concerns that aren't obviously part of the demo. The senior-engineer move is **knowing which ones are real, which ones are in scope, and which ones to defer with an explicit reason**. This section enumerates all three. The deferrals matter as much as the inclusions — they're proof you can see the full production picture even when you choose not to build it.

In scope (build it, don't skip it)

These are not optional. They're hidden inside the existing phases — calling them out so they don't get dropped:

Item	Where it lives	Why it's a blocker
League scoring derivation from <code>league_settings</code>	Part 5 (recommendation engine)	Standard vs PPR vs half-PPR vs TE-premium changes positional value. The scoring function must read from settings, not assume.
Standard roster variants (incl. superflex)	Part 5	The roster_need component is wrong for non-default configs. Superflex (2 QB slots) inverts QB value entirely.
Bye-week distribution awareness	Part 5	Recommendations that load up all RBs in the same bye week are visibly bad to fantasy players.
League_settings versioning	Part 2 (data model)	Settings can change between seasons (keepers especially). Schema must allow it.
Multi-platform ADP (Sleeper + ≥1 other)	Part 3 (Sleeper)	Single-source ADP is a single-market signal. At minimum, blend two.
Live draft-state polling	Part 3 / Part 4	"Live" means detecting opponents' picks within a second or two and reconciling against local state. Import is solved; <i>live</i> sync is a separate load-bearing piece. A tool a pick behind is untrusted mid-draft.

Item	Where it lives	Why it's a blocker
End-to-end latency budget (p95 < ~2s)	Part 5 / Part 6	The user feels sim + reads + news + network, not any one component. Hold the whole pick-to-recommendation pipeline to one number; the LLM explainer stays off the critical path.
Data-source ToS / legal review (one ADR)	Part 3 / Part 11	Sleeper/Yahoo API terms and news-source scraping have ToS implications, especially for a public demo. One ADR documenting what's used, how, and under what terms.
Mobile-responsive war room	Part 4 (UI)	The actual product is used on phones during drafts. Desktop-only fails the demo for half of viewers.
SSE reconnection logic	Part 4 (UI)	A dropped connection mid-draft is a broken product. Reconnect, replay missed events.
Streaming LLM responses	Part 7 (advisor)	A 4-second non-streamed response feels broken. Modern chat UIs stream tokens.
Prompt-injection defense (basic)	Part 7 (advisor)	User input could try to break grounding. At minimum: validator on output, system-prompt reinforcement.
Per-session LLM cost cap	Part 7 (advisor)	Without this, a malicious or unlucky session can rack up real money in token charges.
News deduplication + recency decay	Part 11 (RAG)	Same story across 20 sources should count once; a 3-week-old "questionable" should weigh near zero.
Robust demo seed-data fallback	Part 8 (deploy)	The demo must work when Sleeper is down, when the LLM provider is rate-limiting, when AWS is having a moment. Single most important thing during an interview demo.
Zero-downtime migration discipline	Part 12 (cloud)	Add-column → backfill → make-not-null is the pattern. Document it; practice it once.

Deferred with reason (explicitly out of scope)

These are real production concerns. We are *not* building them. The reason is documented so a reviewer can see we knew what we were skipping and why. Each becomes an ADR in [docs/decisions/](#).

Item	Why it's deferred	Where it would land if undeferred
Keeper leagues	Cost-basis logic (a kept player consumes a draft pick) changes recommendation math; the data model leaves room but the logic isn't built.	Recommendation engine + UI

Item	Why it's deferred	Where it would land if undeferred
Dynasty leagues	Multi-year valuation, rookie picks, future-year draft capital — essentially a different product.	New "dynasty" recommendation track
Auction drafts	Budget allocation problem, not pick-selection problem. Different optimizer entirely.	New "auction" simulator
IDP scoring (defensive players)	The schema supports it; projection model does not. Out of scope until base accuracy is solid.	Projection model + features
Kicker / DEF ML projection	Industry consensus is ML on kickers is fool's gold. Heuristic projection only.	(intentionally not built)
League-mate prior modeling	Cool but data-starved for any individual league. Theoretically possible, not practically valuable.	Opponent model in simulator
Mid-season "orphan team" takeover	Rare workflow, not draft-time.	Roster import + league sync
Feature store (Feast / Tecton)	Over-engineered for current data volume. Simple train/serve consistency tests instead.	Replacing pgvector + repos
Pipeline orchestrator (Airflow / Dagster / Prefect)	Cron + idempotent worker jobs is sufficient at this scale.	Worker process
Multi-region deployment	Single-region in us-east-1 is sufficient; failover documented but not built.	Terraform
Native mobile apps	The PWA + responsive web covers the mobile use case at portfolio scale.	New apps/directory
Multi-tenant / white-label	The product is single-tenant by design.	Auth + data model
Live game-day lineup advice	Different product (start/sit), shares some infra but distinct UX.	New surface
Real-time multi-user collaborative draft view	Two people watching the same draft together. Cool, not core.	SSE + presence layer
Chaos engineering / automated failure injection	Manual failure drills documented in operations.md instead.	CI workflow + chaos tool
Full WCAG AAA	WCAG AA targeted; gaps tracked in an accessibility log.	Frontend

Item	Why it's deferred	Where it would land if undeferred
Multi-agent LLM orchestration	Explicit non-goal. Orchestrator-plus-tools is sufficient (see ADR 0005).	LLM layer

How to use this list during the build

- **Bucket "in scope" items don't get dropped.** As you hit each phase, check the list and make sure the items tagged for that phase land.
- **Bucket "deferred" items get their own one-page ADR.** Each is a `docs/decisions/00XX-deferred-X.md` with: what it is, why we're not building it, what would have to be true to revisit, what changes would be required. These ADRs are part of the resume signal — they prove you understand production complexity even where you chose not to address it.
- **If something comes up that's not on either list:** stop and triage. Either it belongs in scope (and gets added with a target phase), it belongs deferred (and gets an ADR), or it's the user / future-you scope-creeping (kill it).

7. What this proves about you, as a builder

By the time this project is done, the repo demonstrates a specific set of skills. Mapping the work to the signal:

Skill area	The evidence	Where it lives
Product thinking	A working product real users could use	The deployed demo + war room UI
Full-stack engineering	Frontend, backend, DB, cache, worker all talking to each other	The repo as a whole
API design	Versioned, typed, OpenAPI-documented, paginated, error-enveloped	<code>apps/api</code> + the auto-generated docs
Database design	Normalized schema with cross-source ID reconciliation and proper indexes	Alembic migrations + <code>docs/decisions</code>
Algorithm work	Seeded, vectorized, perf-bounded Monte Carlo simulator	<code>services/simulation/</code> + benchmarks in CI
Applied ML	Backtested models that beat a baseline, with calibration and quantile outputs	<code>services/ml/</code> + <code>/backtests</code> public report
MLOps	Experiment tracking, model registry, promotion gates, retraining schedule, drift	MLflow + scheduled workflows

Skill area	The evidence	Where it lives
GenAI engineering	Structured-output extraction feeding deterministic code, grounded tool-calling chat, per-job model tiering (cheap for batch, smart for user-facing), trace logging, token-cost attribution	services/llm/ + worker/ extraction job + admin trace UI
Cloud + IaC	The entire AWS stack provisioned by Terraform	infra/terraform/
DevOps / CI/CD	Automated lint, test, build, deploy on every change	.github/workflows/
Security awareness	Encrypted OAuth tokens, rate limits, secrets management, safe logging	OAuth flow + docs/security.md
Operations	Structured logs, dashboards, alerts, runbooks	Observability config + docs/operations.md
Engineering hygiene	Tests, ADRs, conventional commits, small PRs, self-reviews, postmortems	The repo's history
Communication	README, architecture doc, eval reports, case-study writeup	docs/ + README

A reviewer doesn't need to read this list — they should *feel* each item when they browse the repo.

One honesty note about resume signal. This doc leans on "defensible in an interview" / "resume bullet" framing throughout, and that's a legitimate goal for a portfolio piece. But name the trade-off plainly: a few components (MLflow, drift monitoring, Terraform multi-stack) earn their place *partly* as signal rather than from raw product necessity at this data volume. That's fine — "demonstrates I can operate this machinery" is a real reason. The discipline is to keep it honest: where a component is signal-driven more than product-driven, say so in its ADR rather than retrofitting a product justification. An interviewer respects "I built MLflow to prove I can run a model registry" far more than a strained claim that a rarely-retrained model needed one.

8. The mindset behind the build

A few principles that should color how you make every decision:

Ship the spine, then sharpen each piece. Build the worst possible version of every component first, end-to-end, so the system runs. Then go back and replace each component with the real version. This is the opposite of "build the perfect ML model, then figure out how to use it." It looks slower but finishes ~3x more often.

Treat the LLM as a signal producer or explainer — never an oracle. The LLM has exactly two jobs in this system: (1) extract *structured signals* from unstructured prose (news → typed events) that deterministic code consumes, and (2) *explain* the output of deterministic code in natural language. It never invents numbers. It never ranks players. It never overrides the scoring function. Numerical truth comes from code with seeds, tests, and version numbers. If you ever find yourself wanting the LLM to "just figure out the right pick," stop — that's the moment the product loses credibility. The highest-product-value LLM use in DraftPilot is the silent batch one (news extraction), not the visible chat one; the chat earns its place for

open-ended questions, but the rule is the same in both: structured in, or structured out, never freeform numerical judgment.

Reproducibility over cleverness. Anywhere randomness exists (simulator, model training, sampling), the seed is explicit and tested. Two runs with the same inputs must produce identical outputs. This is *the* thing that separates research code from production code.

The product is the spine. ML is one organ. The project is not "an ML model with a UI bolted on." It's a product that uses ML *as one of several* intelligence layers. ML can be replaced; the product can't. Build the product first.

Boundaries everywhere. Routes don't do business logic. Services don't do HTTP. Repos don't do business logic. The LLM doesn't compute scores. The scoring function doesn't fetch from external APIs. These boundaries are what let each piece be tested and replaced independently.

Budget latency end-to-end, not per-component. This is a *live* draft tool: 60–90 seconds per pick, shared with the user reading, thinking, and acting. The `<800ms` figure on the simulator is a *component* budget, and components lie when you add them up — sim + projection reads + news/risk lookups + the optional LLM explainer + network round-trips is the number the user actually feels. Define a single end-to-end target (recommendation visible in p95 < ~2s from "it's my pick") and hold the whole pipeline to it. The LLM explainer is on the critical path only when clicked, so it must be lazy/streamed, never blocking the core recommendation. If a component can't fit its slice of the budget, it gets cached, precomputed on the prior pick, or degraded — not shipped slow.

Live draft sync is core, not a detail. "Live" means detecting opponents' picks fast enough to react before the user's turn. League *import* (rosters, settings) is solved in the Sleeper phase; live *draft-state polling* — Sleeper's draft API cadence, picking up new picks within a second or two, reconciling against local state — is a distinct, load-bearing piece of the product. Budget for it explicitly in the Sleeper/war-room phases; a tool that's a pick behind is a tool nobody trusts mid-draft.

Demo-able at every step. After every phase, you should be able to send someone a URL and show them something. If a phase doesn't end in a demoable improvement, it was the wrong phase.

The repo is part of the product. Hiring managers don't run your code — they read your repo. A clean PR history, good ADRs, clear docs, and a working CI badge are *part of what you're shipping*. Don't treat them as overhead.

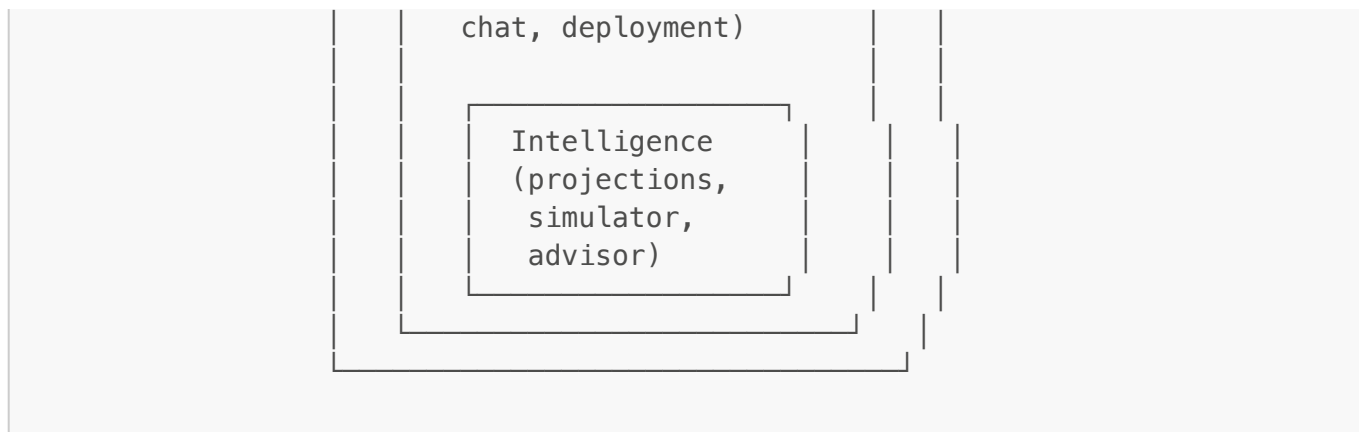
9. A final mental model

Think of DraftPilot AI as **three concentric layers**:

```
graph TD; subgraph Layers; direction TB; L1["Engineering platform<br/>(CI/CD, infra, observability,<br/>tests, docs, security)"]; L2["Product surface<br/>(UI, API, integrations,]"]; L3["Core logic<br/>(LLM, rules, scoring)"]; L1 --- L2 --- L3; end
```

Engineering platform
(CI/CD, infra, observability,
tests, docs, security)

Product surface
(UI, API, integrations,



- The **intelligence layer** is the *what makes it interesting*. It's what differentiates DraftPilot from static rankings.
- The **product surface** is the *what makes it usable*. Without this, the intelligence has nowhere to land.
- The **engineering platform** is the *what makes it credible*. Without this, the project looks like an academic exercise.

All three exist in the final product. The build guide builds them in order from the outside in: platform first (Part 1), then product (Parts 2–4, 7–8), then intelligence (Parts 5–6, 10–11). That's why it feels boring at the start — you're building the layer that *holds the interesting layers* before you build the interesting layers themselves. Trust the order.

TL;DR

DraftPilot AI is a live draft co-pilot whose intelligence is a stack of four layers (projections, scoring, simulation, grounded LLM advisor), wrapped in a real product surface (Next.js + FastAPI + Postgres + Redis), running on a real engineering platform (AWS + Terraform + CI/CD + tests + observability). Each layer answers a specific question the user has. Each component in the architecture earns its place by serving a user-facing need. The build order goes platform → product → intelligence, because that order maximizes the chance you finish — and because the intelligence layers only make sense once there's a product to put them in.

Now read the build guide and start at Phase 1.